JavaScript 事件循环到底在干什么？

一句话：

JavaScript 是**单线程**的，事件循环是浏览器 / Node 用来**安排“现在执行什么、稍后执行什么”**的一套机制，让异步代码看起来像是“以后再执行”。

基本概念

- **调用栈（Call Stack）**
  - **同步代码**一行一行进栈执行，执行完出栈。
  - 栈为空时，事件循环才有机会把“排队的回调”压进来执行。
- **任务队列（Task Queue / Callback Queue）**
  - 放 **宏任务（macrotask）**：比如 `setTimeout` 、 `setInterval` 、DOM 事件回调、 `XMLHttpRequest` 回调等。
  - 当调用栈清空后，事件循环会从宏任务队列取出一个任务执行。
- **微任务队列（Microtask Queue）**
  - 放 **微任务（microtask）**：比如 `Promise.then/catch/finally` 、 `queueMicrotask` 、 `MutationObserver` 。
  - **每执行完一个宏任务之后，立即把当前所有微任务执行完**，然后再去取下一个宏任务。

典型执行顺序（浏览器中）

1. 执行全局同步代码（这是第一个宏任务的一部分）。
2. 遇到 `setTimeout` ：把回调丢进**宏任务队列**（等时间到了再排队）。
3. 遇到 `Promise.then` ：把回调丢进**微任务队列**。
4. 全局代码执行结束 → 调用栈清空。
5. 事件循环：
   - **先清空所有微任务队列**（FIFO，依次执行 `then` 等）。
   - 微任务执行过程中如果又产生新的微任务，继续加到本轮微任务队列末尾，本轮都要执行完。
   - 微任务队列清空后 → 取出一个宏任务执行（比如某个 `setTimeout` 回调）。
   - 该宏任务执行完 → 再次执行它产生的所有微任务 → 再取下一个宏任务……

一个常见例子

```
console.log('start');

```

```
setTimeout(() => {
  console.log('timeout');
}, 0);

Promise.resolve().then(() => {
  console.log('promise');
});

console.log('end');

```

执行顺序：

1. `console.log('start')`
2. 注册 `setTimeout` 回调（放到宏任务队列）
3. 注册 `Promise.then` 回调（放到微任务队列）
4. `console.log('end')`
5. 全局代码结束 → 调用栈为空
6. 开始清空微任务队列 → 打印 `promise`
7. 微任务队列清空 → 取出宏任务 → 执行 `setTimeout` 回调 → 打印 `timeout`

最终输出顺序： `start` → `end` → `promise` → `timeout`

Node.js 中的事件循环（简单对比）

- Node 也有事件循环，但基于 **libuv**，分了更多阶段（timers、I/O callbacks、check 等）。
- 但核心规律类似：
  - **宏任务**： `setTimeout` 、 `setImmediate` 、I/O 回调等（按不同阶段排队）。
  - **微任务**： `process.nextTick` 、 `Promise.then` 。
- 一个阶段里：执行回调 → 清空微任务 → 进入下一个阶段。